Microsoft

January 28th, 2026

Network Blackbox Trace Analysis using AI & Data Visualization DevOps Tool

Tung V. Le
Software Engineer Intern, Azure for Operators

Special thanks:

Vinay Cherian Jaeyong Yoo Lincoln Xiong

Network BlackBox Trace Analysis using AI & Data Visualization DevOps Tool

Tung V. Le
Software Engineer Intern, Azure for Operators (Data-plane Team)

Problem

- Sometimes, Azure's clients measure a performance regression. They want an explanation.
- Most of the time, it is not our system responsible. FTE engineers spend lots of time and effort to investigate blackbox log and come up with an explanation behind the system's decision.

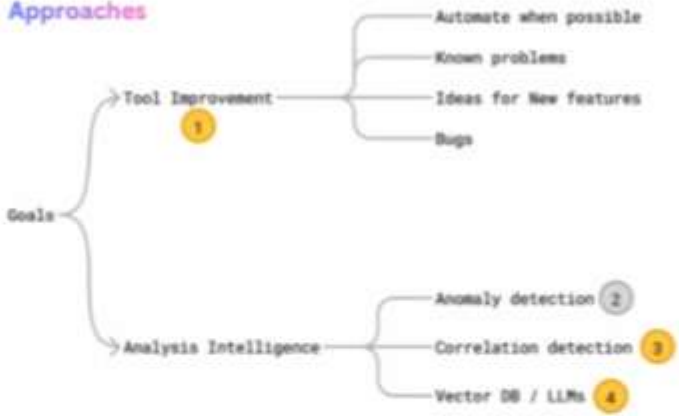
Objectives

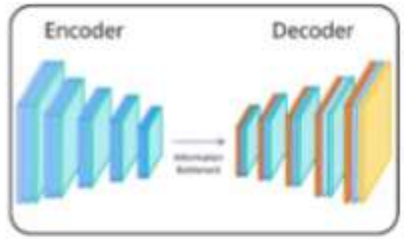
1. Reduced time and effort require for manual network analysis by automating parts of the process where possible.
2. Reduced time and effort to get blackbox trace understanding
3. Added features for developer productivity and visualization blackbox tool.
4. Foundation for AI assist integration and automation

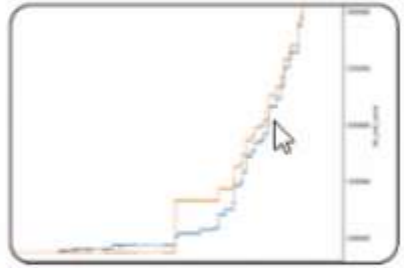
Future Work

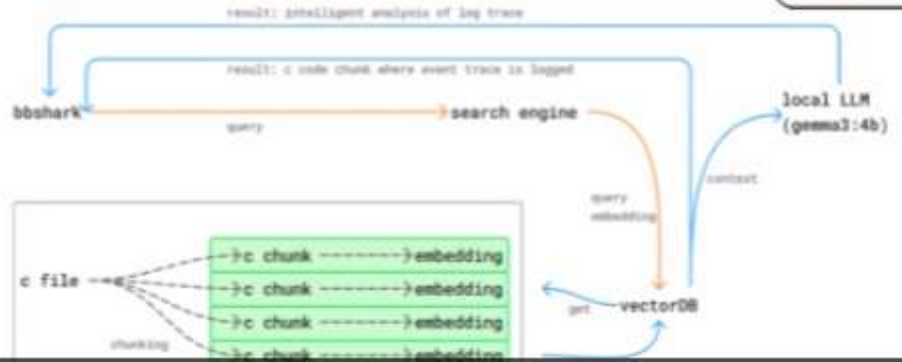
- larger-context AI models, automated decision explanations, richer correlation and causal analysis, and code-aware reasoning: from single-trace insight to system-level understanding.

Approaches

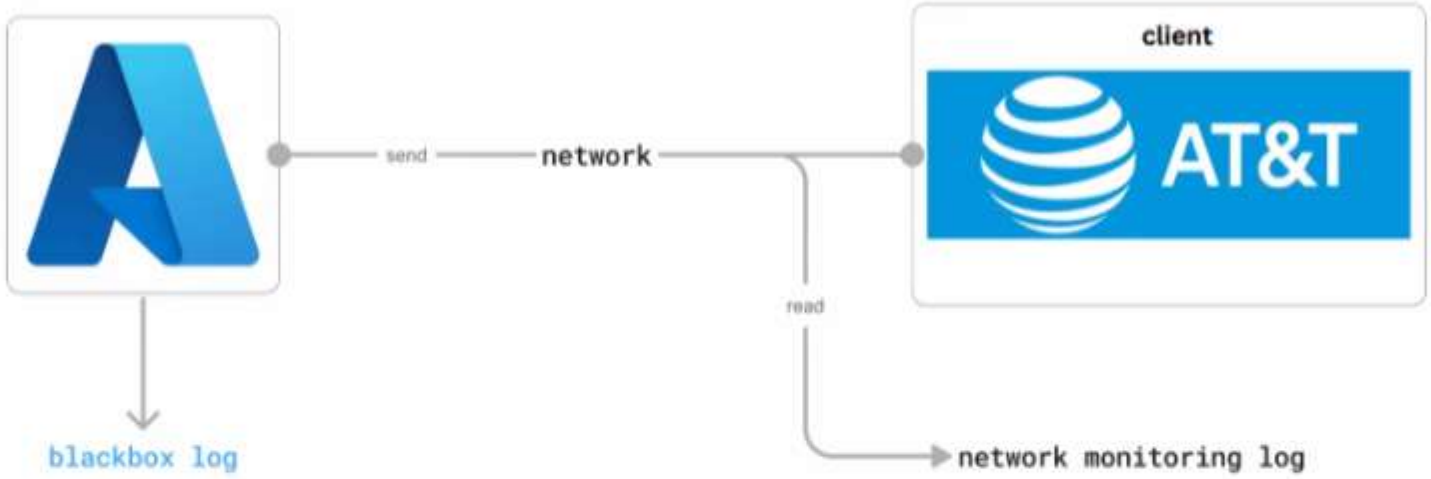






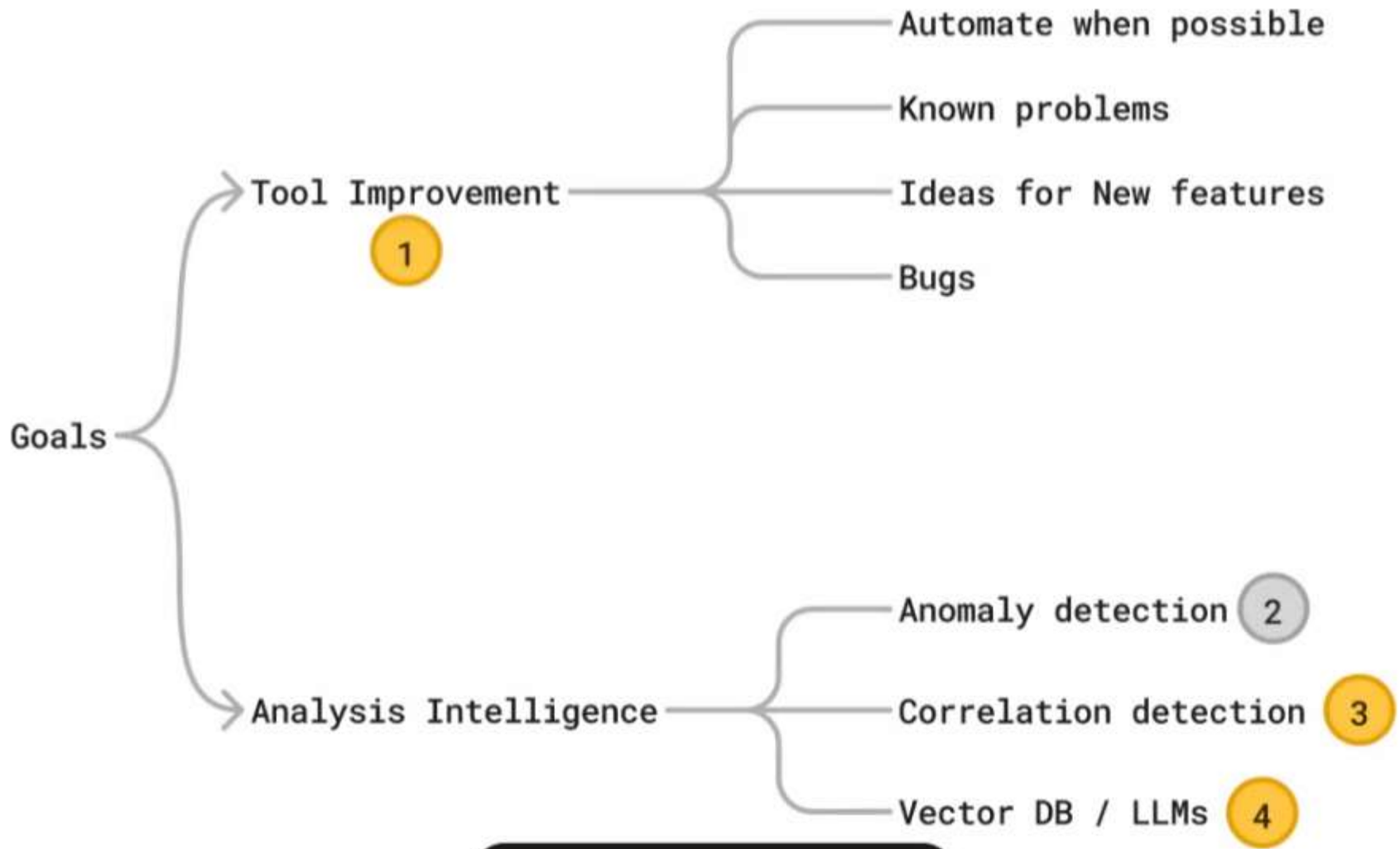


Background Context



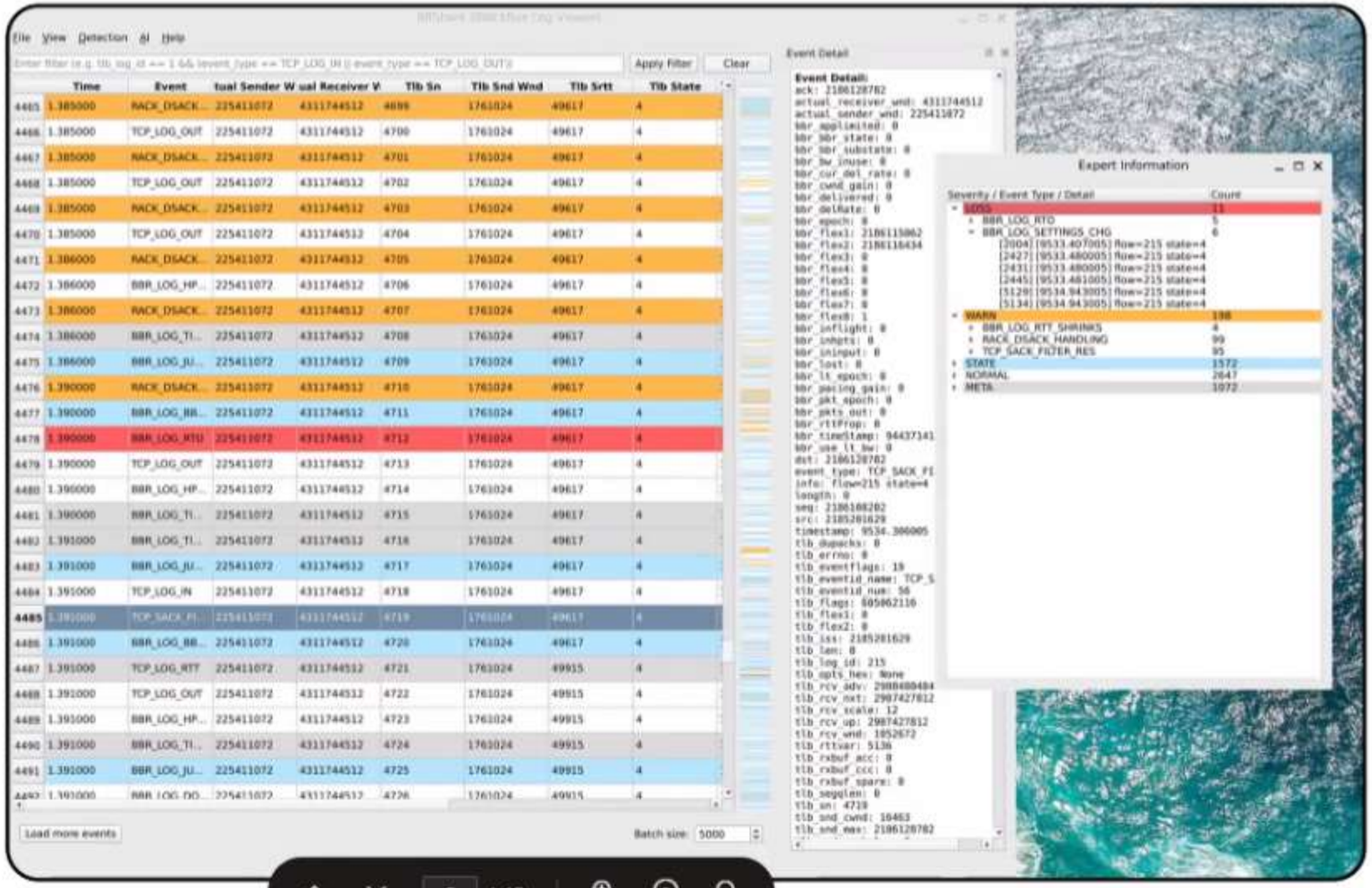
The focus should be answering the question of “Why our system made this decision?”, “What happened?” rather than “What went wrong?”

Provide insights/information for client (operators). The goal is to have a tool that **1.** help engineer quickly gather information and **2.** Automatically and intelligently provide explanation of what happened.


Goals
Tool Improvement
1
Automate when possible
Known problems
Ideas for New features
Bugs
Analysis Intelligence
Anomaly detection
2
Correlation detection
3
Vector DB / LLMs
4

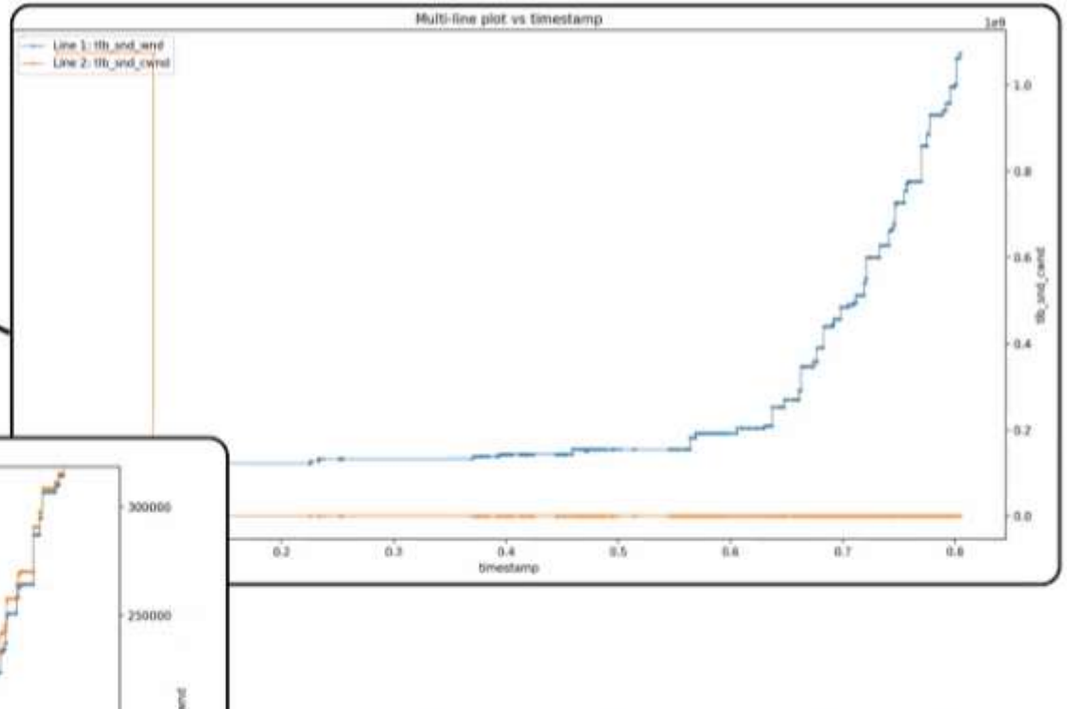
1

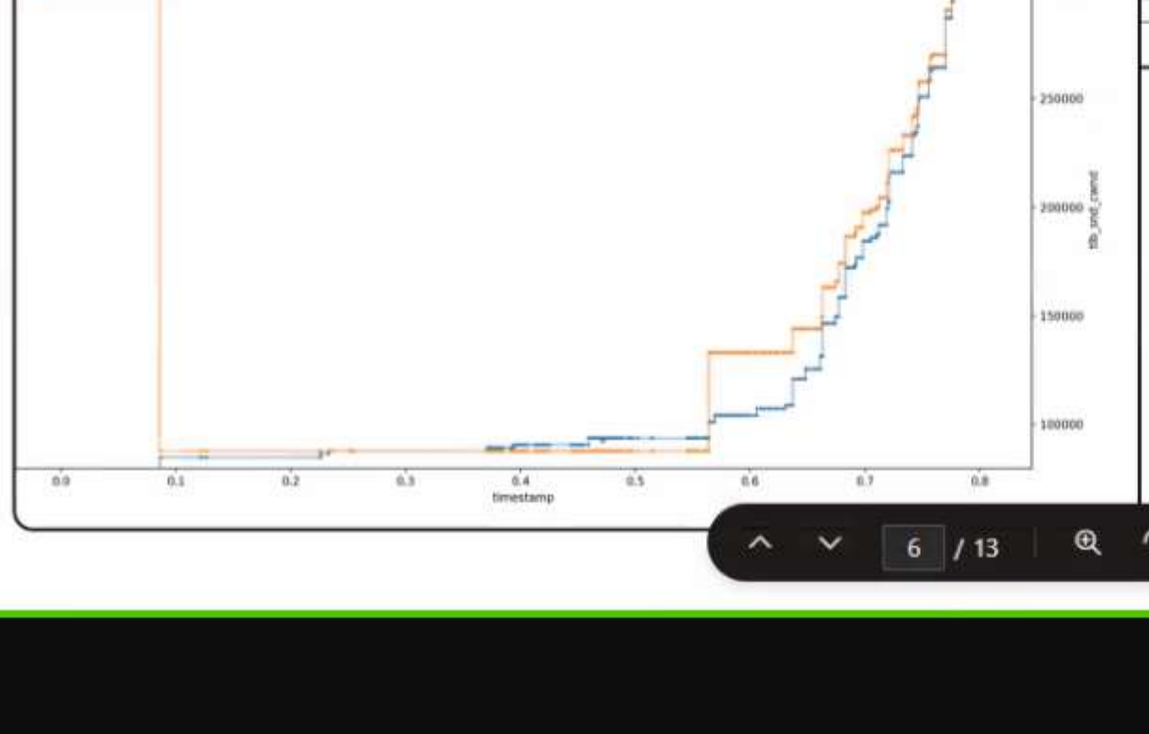
DevOps Tool Improvements



Graphical Improvements

Orange Line: featureless since it has extreme value → impossible to tell the correlation between 2 plots





Problem: Graph non-interactive (redraw on every click), unable to see plot features with extreme values, manual set up
→ Solution: added interactive graph, automatically selected Y limits and **quick plot** feature for 2 lines for comparison.

Solved Problem → Solution List

- (*) Mismatching Event Attributes: Same logical events appear with inconsistent fields across logs → ConfigMap-based event normalization to reconcile attributes during ingestion. Later use search DB.
- (*) Large Log Files Take Forever to Load: Full file loading causes long waits with no scope control → Lazy batch loading with user-controlled dynamic batch size
- (*) Filters, highlights, and context are lost when reopening files → Save and restore full graph context including filters, view state, and annotations
- (*) Hard to Visually Scan Event Patterns: Dense logs make anomalies and transitions difficult to spot → Color-coded event types, categorized timelines, expert information panels, hierarchical views
- Graph non-interactive (redraw on every click), unable to see plot features with extreme values, manual set up → added interactive graph, automatically selected Y limits and quick plot feature for 2 lines for comparison.
- Limited Post-Processing Capability: Built-in metrics insufficient for ad-hoc analysis → Excel-style custom value columns with user-defined calculations
- Slow Filtering Workflow: Manually writing filters disrupts investigation flow → Context-menu driven filtering with one-click rule creation
- Silent Filter: Invalid predicates fail quietly and confuse users → Grammar-aware filter validation with visual warnings
- Reopening large traces interrupts iterative debugging → Save/load analysis state for faster workflow iteration
- No existing per-flow analysis → Dedicated per-flow analysis mode with isolated timelines and metrics
- UI Freezes During Heavy Computation: AI search, correlation, and anomaly detection block interaction → Non-blocking background execution with multitasking support
- UI logic tightly coupled with data processing slows iteration → between GUI and data layers

(*) inital goal achieved
stretch goal achieved

2

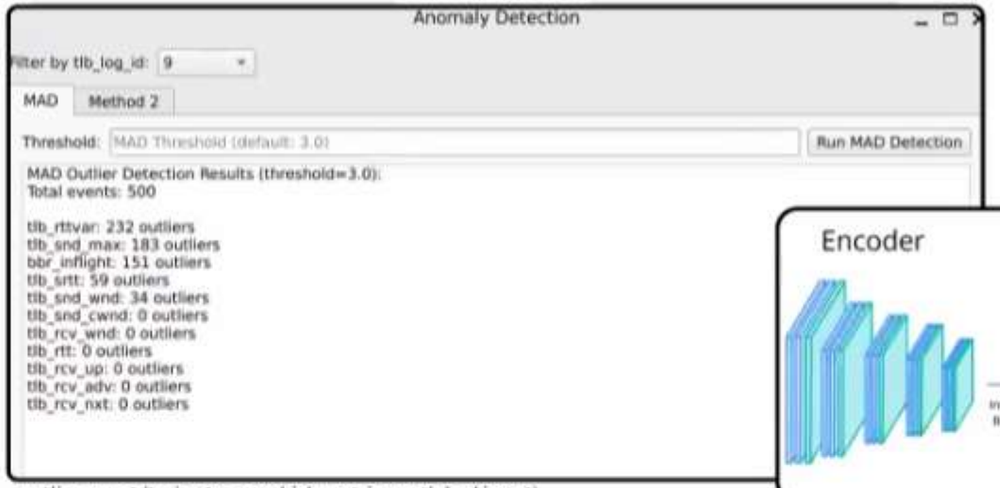
Analysis Intelligence Approach #1

Anomaly Detection

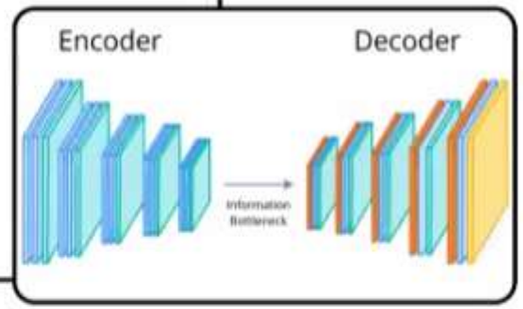
Identify the most "anomalous" metrics using outlier counts (MAD) or unsupervised models (autoencoders) to flag unusual behavior.

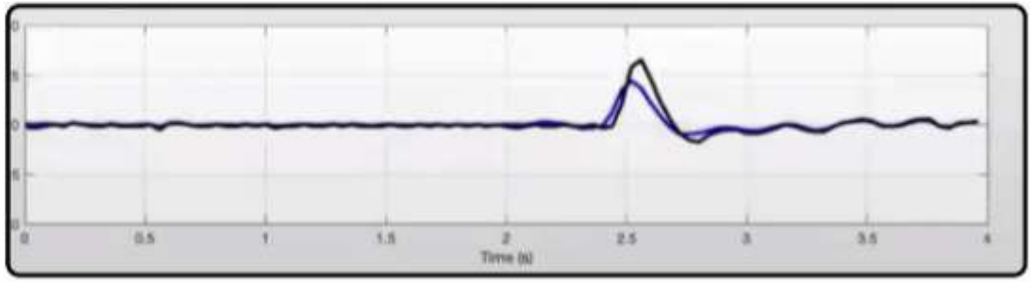
Pros: Unsupervised, no need label, plenty of 'normal' data
Cons: Anomalies are non-deterministic and hard to validate. Models learn statistical deviation, not intent - there is no absolute "normal."

Why I stop exploring this? The real value is not labeling behavior as abnormal, but explaining why the system made a specific decision at that moment, not find 'errors', since it is hard to say 'faulty' or 'abnormal'.

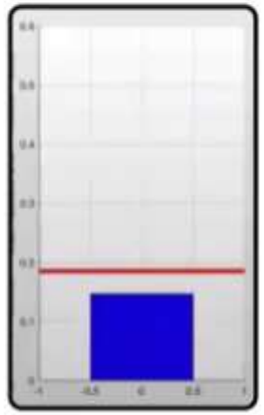


outlier count (trying to see which metric worth looking at)





Actual behaviour vs expected behaviour



Error threshold

3

Analysis Intelligence Approach #2

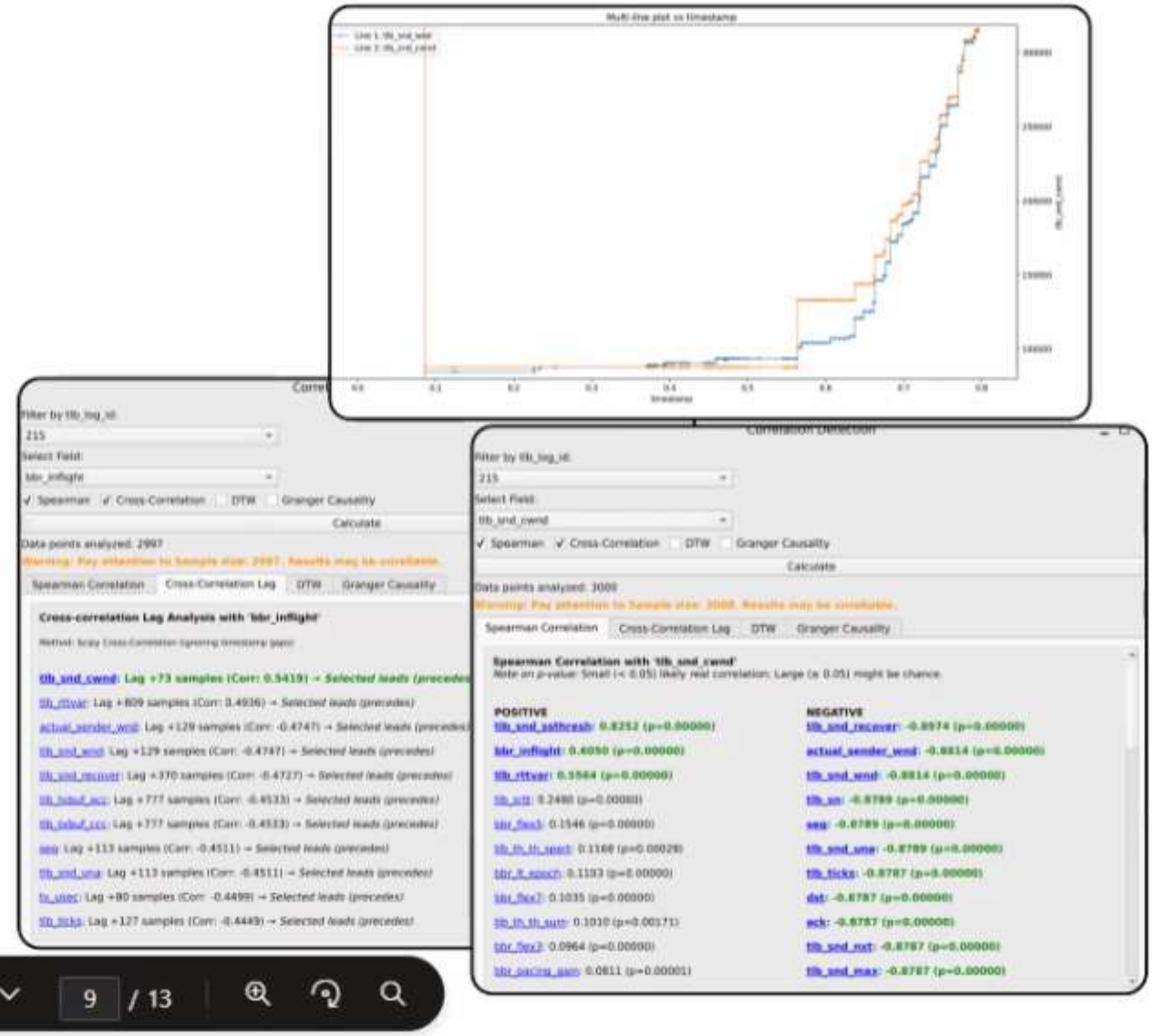
Correlation Detection

Identify metrics of interest among hundreds by uncovering meaningful relationships: linear, monotonic, lagged, and shape-based. Correlation analysis helps reason about cause-effect behavior, explain why a metric changed, and pinpoint when interactions occurred.

- **Supported:** p-value - quantify statistical confidence vs random coincidence
- Spearman correlation - monotonic relationships beyond linear assumptions
- Cross-correlation (lag) - temporal lead-lag behavior for causal reasoning
- Granger causality - infer directional influence between metrics
- Dynamic Time Warping (DTW) - similar patterns with skewed or uneven timing

Benefits

Determinisitc, transforms raw metrics into interpretable relationships that explain behavior that help FTE engineers get more information faster.



4

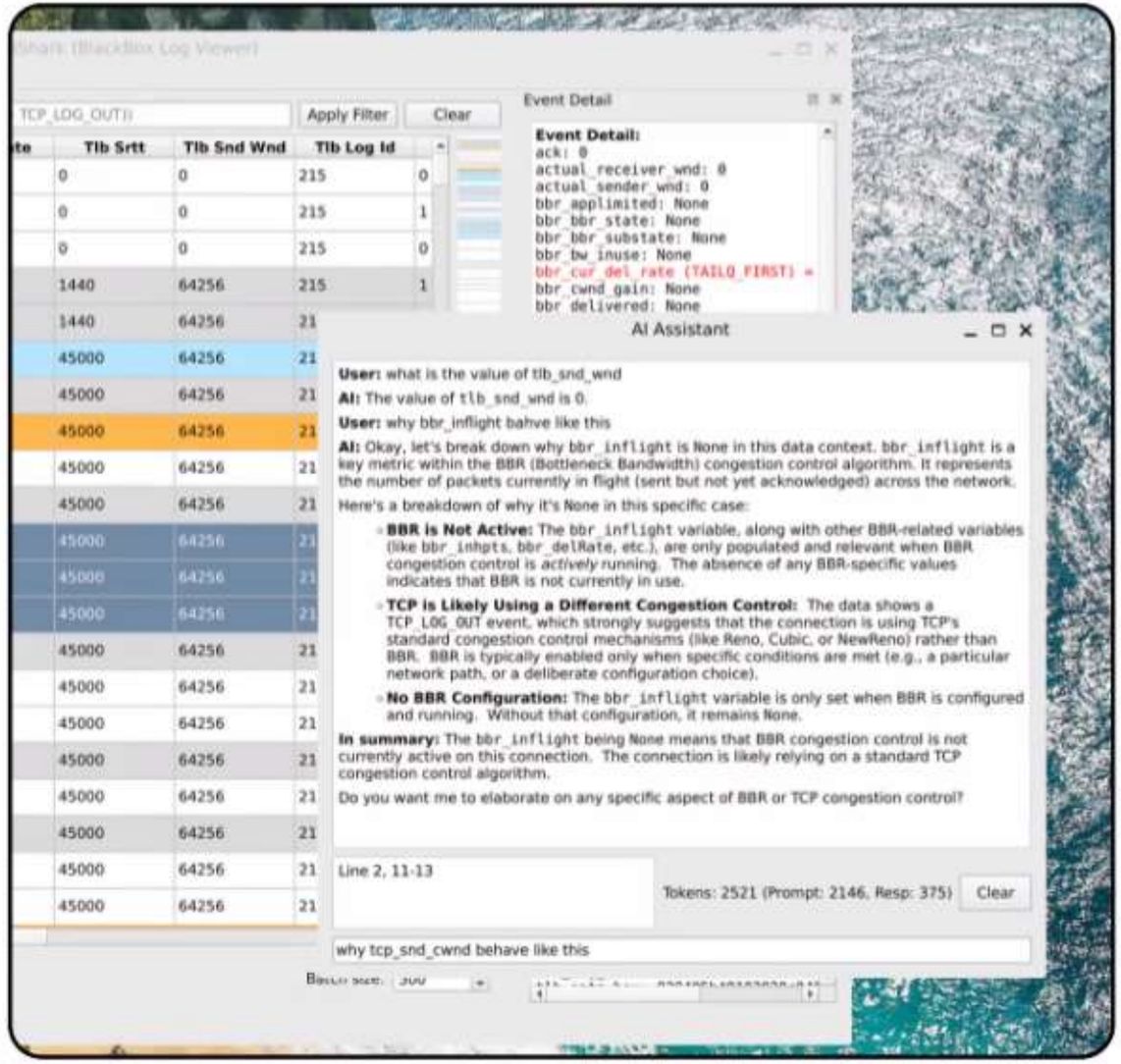
Analysis Intelligence Approach #3

Vector DB & LLM

My goal was simple: **can we quickly explain what's happening behind the scenes?**
What if I give an LLM enough context—both the code and a few trace events—could it explain what happened, what decision was made, and why?

The first challenge was model choice.
Because the data is proprietary, cloud LLMs (e.g., Azure OpenAI) were risky and expensive to experiment with. I pivoted to local open-source models—fully private, but with trade-offs in memory, speed, and model size. After experimentation, I chose Gemma 3 (4B) as a practical balance: not the best model, but fast and usable on my machine.

The next problem was context size.
The C codebase is far too large to pass directly to an LLM. Instead of forcing everything into context, I changed the approach.
I chunk the code by function, isolating logical units where specific events are logged.
Each chunk is indexed into a vector database, so when I search for an event type, I get the exact code context that produced it—no more, no less.



This is better than:

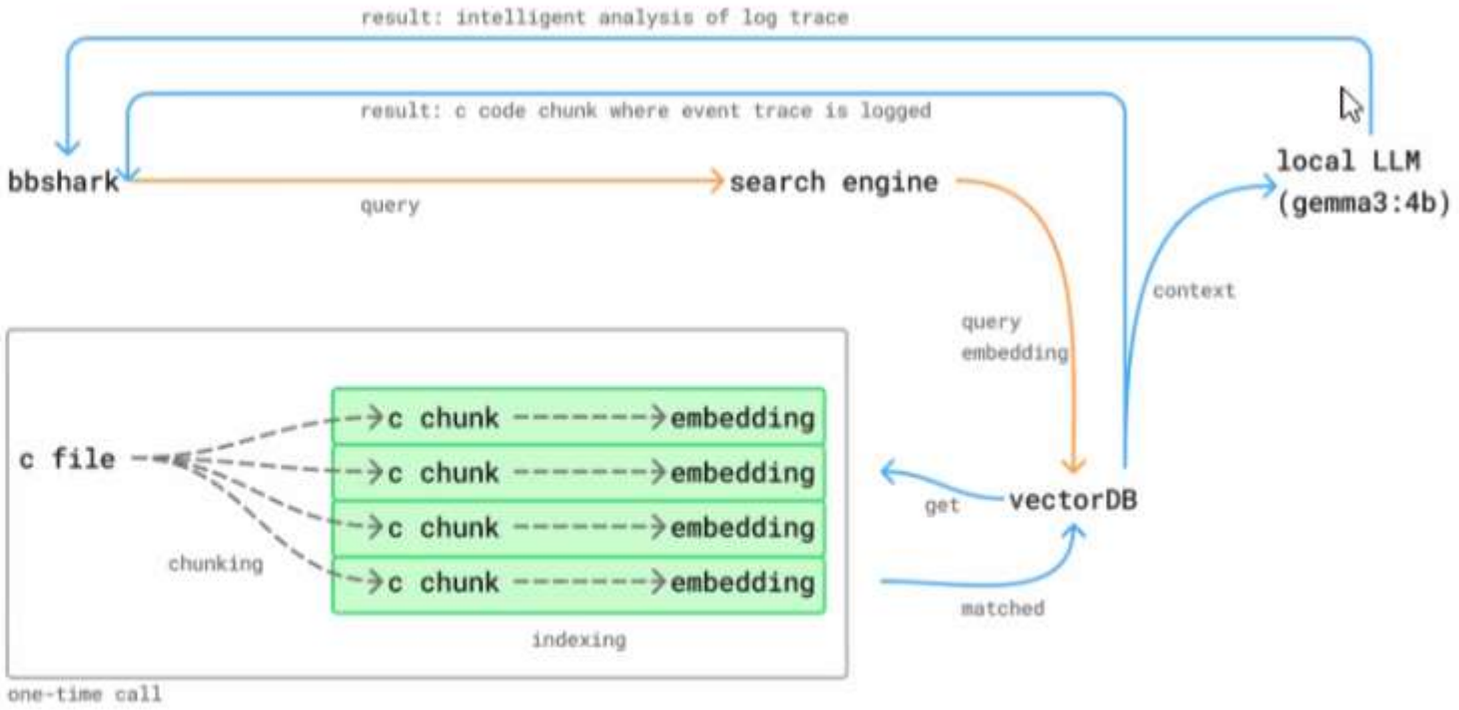
- keyword search (only gives a line, no logic), and
- asking the LLM directly (slow, non-deterministic, hard to verify).

If an event is logged in multiple places, I disambiguate using attributes like line number, which uniquely identifies the logging site. This narrows the problem to the exact decision point in code.
Now, instead of giving the LLM a massive codebase, I give it one small, precise chunk.
At that point, the LLM can reliably explain the logic, step by step, tied directly to the trace event.

This approach is clean, scalable, and low-maintenance:

- No manual config maps
- Automatically adapts to code changes (just re-index)
- Deterministic search + explainable AI

So now, when someone asks "what's going on behind the scenes?", they can either read the exact code, or ask the LLM to explain what happened and why, with confidence.



	Manual Config Maps	Pure LLM (Full Context)	Keyword / Grep Search	**Proposed: Vector Search + LLM**
Determinism	❌ Manual, error-prone	❌ Non-deterministic	✅ Deterministic	✅ Deterministic search, explainable output
Scalability	❌ Hard to maintain as code grows	❌ Context size limits	✅ Scales well	✅ Scales to large codebases
Context Accuracy	⚠️ Depends on human correctness	❌ Hallucination risk	❌ No semantic meaning	✅ Exact code chunk, precise scope
Handling Multiple Log Sites	❌ Difficult to disambiguate	❌ Unreliable reasoning	❌ Returns many matches	✅ Attribute-based disambiguation
Explainability	❌ Static mapping only	⚠️ Hard to trust	❌ None	✅ Line-by-line code explanation
Adaptability to Code Changes	❌ Requires rework	⚠️ Re-prompting needed	✅ No setup	✅ Re-index once, auto-adapts
Performance	✅ Fast lookup	❌ Slow, expensive	✅ Very fast	⚠️ Fast search, medium speed
Maintenance Cost	❌ High	❌ High	✅ Low	✅ Low, mostly automated

Microsoft

Conclusion & Future Work

- Added features for developer productivity and visualization
- Reduced time and effort require for manual network analysis by automating parts of the process where possible.
- Reduced time and effort to get blackbox trace understanding
- Foundation for AI assist integration and automation

Future Work:
Larger-context AI models, automated decision explanations, richer correlation and causal analysis, and code-aware reasoning: from single-trace insight to system-level understanding.

Thank you!